

Виртуальная память – распространенная стратегия распределения памяти, используемая во всех современных операционных системах, основанная на идее расширения физической памяти путем размещения расширенной памяти на диске и использования таблиц страниц (или сегментов) для трансляции адресов. В лекции рассмотрены следующие вопросы:

- Мотивировка концепции виртуальной памяти;
- Потребность в страничной организации;
- Создание процесса и его пространства виртуальной памяти;
- Замена страницы;
- Размещение фреймов;
- Thrashing;
- Примеры организации виртуальной памяти в различных ОС.

Концепция **виртуальной памяти** основана на идеях отделения логической памяти пользователя от физической памяти и расширения логической памяти путем хранения ее образа на диске.

При исполнении программы только часть ее кода и данных, к которым происходит обращение, в каждый момент требует размещения в физической памяти. Поэтому, естественно, возникает идея расширить пространство логической памяти, которое может быть реализовано намного большего размера, чем физическая память. Это и есть основной принцип организации виртуальной памяти.

Виртуальная память поддерживает совместное использование одного и того же адресного пространства более чем одним процессом, создание и исполнение облегченных процессов в общем пространстве виртуальной памяти.

Виртуальная память допускает более эффективное создание процесса, чем предшествующие схемы организации памяти и процессов.

Заметим, что концепция виртуальной памяти непосредственно не связана ни со страничной, ни с сегментной стратегиями распределения памяти. Виртуальная память может быть реализована различными способами, например, с помощью:

- страничной организации по требованию (paging on demand);
- сегментной организации по требованию (segmentation on demand).

В приведенных терминах подчеркивается динамический характер управления виртуальной памятью: термин **по требованию** означает, что страница или сегмент будут размещены в физической памяти только в случае, если к ним реально происходит обращение из программы пользователя. Причем если размер обрабатываемой области виртуальной памяти (например, массива) очень велик – например, 1000 страниц, то в физической памяти будет размещена только та его страница, к которой обращается пользовательская программа.

Страничная организация по требованию

Принцип реализации виртуальной памяти в виде **страничной организации по требованию** заключается в том, что каждая страница загружается в память, только если она реально требуется при выполнении программы – содержит код или данные, к которым произошло обращение.

Преимущества данного подхода:

– **Меньший объем ввода-вывода:** В память подкачивается только минимально необходимый объем данных (например, одна страница большого массива, а не весь многостраничный массив);

– **Меньший объем памяти:** При данном способе расходуется минимально необходимый объем физической памяти;

– **Более быстрая реакция системы:** Поскольку объем пересылаемых данных меньше, система в среднем быстрее реагирует на каждый запрос к памяти;

– **Система может обслуживать большее число пользователей:** Ввиду экономии физической памяти и времени обращения, система в состоянии при данном подходе обслуживать большее число пользовательских процессов.

Основные принципы страничной организации по требованию:

1. Если страница требуется программе, на нее имеется ссылка из программы.
2. Если ссылка на страницу неверна (например, страницы с данным номером не существует), происходит прерывание.
3. Если требуемая страница отсутствует в памяти, то она подкачивается в память. Механизм подкачки реализуется через прерывание (page fault – отказ страницы).

С каждым элементом таблицы страниц связывается бит "**valid/invalid**", однако, в отличие от организации логической памяти, он играет несколько иную роль – он указывает на присутствие или отсутствие страницы в основной памяти. Значение бита равно 1, если страница в памяти, и 0, если (страница отсутствует в памяти).

Первоначально для всех элементов таблицы страниц бит valid/invalid полагается равным 0.

Если в процессе трансляции адреса бит "**valid/invalid**" в таблице страниц окажется равным 0, то происходит **прерывание по отсутствию страницы в памяти (page fault)**.

Обработка ситуации отсутствия страницы в памяти

Если в таблице страниц имеется ссылка на страницу, отсутствующую в памяти, первое же обращение по такой ссылке приведет к прерыванию и вызову ОС (ситуации page fault – отсутствие страницы в памяти)

ОС по таблицам определяет, что именно произошло:

Если имеет место неверная ссылка (на страницу, отсутствующую в логической памяти), то работа программы прекращается.

Если же имеет место обычное отсутствие страницы в памяти, то ОС должна разместить его в основной памяти. Для этого ОС выполняет следующий алгоритм:

- Найти незанятый фрейм в основной памяти;
- Считать содержимое страницы в данный фрейм;
- Изменить элемент таблицы страниц: validation-бит установить равным 1;
- Продолжить работу программы.

Напомним, что программа после прерывания продолжается с **той же** команды, которая была прервана из-за отсутствия страницы. Поэтому теперь программа продолжит нормально выполняться, и обращение к странице произойдет успешно.

Преимущества виртуальной памяти при создании процессов

Благодаря механизму виртуальной памяти, могут быть использованы следующие оптимизации расходования памяти при создании процессов:

- Копирование по записи (Copy-on-Write)
- Отображение файлов в память (Memory-Mapped Files).

Принцип совместного использования страниц процессами (или копирование по записи - Copy-On-Write, COW) позволяет первоначально родительскому и дочернему процессам использовать одни и те же страницы памяти. Если какой-либо процесс модифицирует разделяемую страницу, то только в этом случае данная страница копируется. Принцип COW обеспечивает более эффективное создание процесса, так как копируются только модифицируемые страницы. Свободные страницы распределяются из списка страниц, инициализированных нулями.

Использование при вводе-выводе файлов, отображаемых в память, позволяет рассматривать файловый ввод-вывод как обычное обращение к памяти путем отображения блока на диске в страницу памяти.

Первоначально файл читается с использованием запроса страниц по требованию. Часть файла размером с одну страницу читается из файла в физическую страницу (фрейм). Последующие чтения из файла и записи в файл трактуются как обычные обращения к памяти. Это упрощает доступ к файлу, по сравнению с системными вызовами `read()` и `write()`. Это позволяет также нескольким процессам отображать в память один и тот же файл, по тому же принципу, как они совместно используют какие-либо страницы.